

Recuperacion de Bundle a Producto en Moda: Paper Tecnico y Documentacion del Sistema

Equipo Hack2026
Informe Tecnico Interno

Abstract—Este documento describe el sistema completo de retrieval de Hack2026 para descomposicion de bundles de moda, donde cada imagen de bundle debe asociarse con uno o mas identificadores de producto. La implementacion actual se apoya en OpenCLIP (Marqo Fashion SigLIP), con deteccion opcional de regiones de ropa y varias estrategias de inferencia (pipeline principal, per-crop v8, phase1 por seccion y top-1 por crop). Se detallan el preprocesamiento de datos, el aumento offline, el entrenamiento del modelo, la validacion local, la inferencia y la generacion de entregas. El paper se apoya en el codigo del repositorio, con enfasis en reproducibilidad y decisiones de ingenieria.

Index Terms—recuperacion visual, aprendizaje multimodal, OpenCLIP, IA en moda, metric learning, matching a nivel de objeto

I. INTRODUCCION

La tarea Hack2026 se formula como retrieval visual multietiqueta: dada una imagen de bundle, el sistema debe recuperar los `product_asset_id` correctos y ordenar hasta 15 candidatos por bundle para evaluacion.

La base actual del repositorio es OpenCLIP (Marqo Fashion SigLIP) con varias rutas de inferencia especializadas. A nivel de ingenieria, el flujo de trabajo se divide en:

- 1) Preparacion de datos e imagenes (`download.sh`, `preprocess_data.py`, `offline_augment.py`).
- 2) Entrenamiento de retrieval multimodal (`python -m src.train`, backend `src/models/retrieval_openclip.py`).
- 3) Inferencia y exportacion de submission (`src.infer`, `src.new_infer`, `src.infer_phase1`, `src.infer_top1`, `src.infer_openclip`).

Adicionalmente, el proyecto incluye scripts de analisis para explotar senales de metadata temporal y estructura categoria-seccion (por ejemplo, `src/utlis/check_link_timestamps.py`).

Este documento resume la implementacion vigente y los artefactos observables del workspace.

II. DATOS Y PREPROCESAMIENTO

A. Datasets de Origen

Las entradas principales son cuatro CSV. Su rol funcional se resume en la Tabla ??.

Archivo	Proposito
<code>data/bundles_dataset.csv</code>	Catalogo de bundles y URLs de imagen.
<code>data/product_dataset.csv</code>	Catalogo de productos, URLs y descripcion textual.
<code>data/bundles_product_mappings_train.csv</code>	Entrenamiento supervisados bundle-producto para entrenamiento.
<code>data/bundles_product_mappings_test.csv</code>	Entrenamiento no supervisados bundle-producto para predecir <code>product_asset_id</code> .

TABLE I
DATASETS USADOS POR ENTRENAMIENTO E INFERENCIA.

B. Snapshot Local (2026-02-28)

Inspeccion directa del workspace:

- Bundles en catalogo: 2,331.
- Productos en catalogo: 27,688.
- Relaciones train: 6,493 (1,876 bundles unicos, 4,012 productos unicos).
- Filas test: 455 (sin IDs de bundle fuera del catalogo).
- Imagenes locales: 2,331 bundles y 27,688 productos.

C. Pipeline de Preprocesamiento

`preprocess_data.py` implementa validacion de esquema, chequeos de integridad referencial, split train/val por `bundle_asset_id`, descarga opcional de imagenes y generacion de manifiestos. Artefactos principales:

- `data/manifests/train_manifest.jsonl`
- `data/manifests/val_manifest.jsonl`
- `data/labels.json`, `data/label2idx.json`, `data/stats.json`

En el estado actual, `data/stats.json` reporta 86 etiquetas limpias y 6,493 relaciones validas tras limpieza.

D. Aumento Offline

`offline_augment.py` genera vistas sinteticas para bundles y productos con semillas deterministas por `asset+indice` de augmentation. El script soporta manifiests CSV o JSONL y guarda:

- `bundles_aug/*.jpg`, `bundles_aug_manifest.jsonl`
- `products_aug/*.jpg`, `products_aug_manifest.jsonl`

La motivacion principal es robustecer embeddings ante la restriccion de pocas vistas por producto.

III. METODOLOGIA

A. Formulacion del Problema

Para cada bundle b , se ordena un conjunto grande de productos $\{p_i\}_{i=1}^N$ y se devuelven hasta 15 IDs unicos. El entrenamiento usa enlaces positivos de `bundles_product_match_train.csv`.

B. Representacion Multimodal con OpenCLIP

El encoder base es Marqo Fashion SigLIP. Para productos, el sistema puede fusionar embedding visual y embedding textual de `product_description`; para bundles se usa principalmente embedding visual de imagen/crops.

La similitud se calcula con coseno (producto punto tras normalizacion L2):

$$\text{sim}(\mathbf{q}, \mathbf{P}) = \mathbf{q}\mathbf{P}^\top \quad (1)$$

C. Entrenamiento Contrastivo

`src/models/retrieval_openclip.py` optimiza una perdida contrastiva bidireccional (bundle $\leftrightarrow$ product) con positivos multiples por bundle dentro del batch, scheduler coseno con warmup y AMP opcional.

D. Inferencia Guiada por Deteccion

Cuando `params.use_bundle_boxes=true`, se detectan regiones de ropa con YOLO y se hace retrieval por crop. Si no hay detecciones, se usa fallback de imagen completa. La cache de cajas se guarda en JSON para reutilizacion entre corridas.

E. Post-procesado

Segun script/config, se aplican:

- filtro de genero (bundle section vs. `product_dataset_with_gender.csv`);
- diversidad por categoria (maximo por `product_description`);
- umbral minimo de score;
- reranking por cercania temporal (timestamp `ts` en URLs).

F. Variantes de Inferencia

El repositorio mantiene varias estrategias:

- `src.infer`: pipeline principal OpenCLIP+YOLO+filtros.
- `src.new_infer`: per-crop retrieval con TTA y FAISS opcional.
- `src.infer_phase1`: filtrado por seccion + zero-shot de categoria por crop.
- `src.infer_top1`: top-1 por crop para predicciones mas compactas.
- `src.infer_openclip`: version simple por argparse.

G. Metricas

La validacion local usa Hit@K y Recall@K (`src/utils/metrics.py`). Para un bundle con verdad terreno G_b y top-K predicho $\hat{G}_b^K$:

$$\text{Recall@K}(b) = \frac{|G_b \cap \hat{G}_b^K|}{|\hat{G}_b^K|} \quad (2)$$

IV. DETALLES DE IMPLEMENTACION

A. Configuracion y Entrypoints

El proyecto usa Hydra con dataclasses tipadas (`src/config.py`). Entrypoints principales:

- `python -m src.train`: entrenamiento OpenCLIP.
- `python -m src.infer`: inferencia principal (salida en directorio Hydra).
- `python -m src.new_infer`: inferencia v8 (salida en `outputs/`).
- `python -m src.infer_phase1`: inferencia con seccion + zero-shot.
- `python -m src.infer_top1`: inferencia top-1 por crop.
- `python -m src.infer_openclip`: inferencia simple por argparse.

B. Mapa de Modulos

Modulo	Responsabilidad
<code>src/train.py</code>	Bootstrap Hydra y despacho de backend de entrenamiento.
<code>src/models/retrieval_openclip.py</code>	Optimiza train/val, checkpoints y metricas JSONL.
<code>src/infer.py</code>	Inferencia principal con deteccion opcional y post-procesado configurable.
<code>src/new_infer.py</code>	Variante per-crop + TTA + indice FAISS opcional.
<code>src/infer_phase1.py</code>	Variante con indices por seccion y zero-shot de categoria.
<code>src/infer_top1.py</code>	Variante top-1 por crop para salidas compactas.
<code>src/infer_openclip.py</code>	Script legacy de inferencia con argumentos CLI.
<code>src/detection.py</code>	Wrapper de detector YOLO clothing y utilidades de cajas.
<code>src/utils/metrics.py</code>	Metricas Hit@K y Recall@K.
<code>src/utils/add_gender.py</code>	Generacion de <code>product_dataset_with_gender.csv</code> .
<code>src/utils/check_link_timestamp.py</code>	Revisita de consistencia temporal bundle-producto.

TABLE II

MODULOS PRINCIPALES Y RESPONSABILIDADES.

C. Notas Operativas Importantes

- 1) `src/train.py` espera por defecto `data/product_dataset_with_gender.csv` como manifiesto de productos.
- 2) Si CUDA no esta disponible y se solicita `device=cuda`, el codigo hace fallback a CPU en varios entrypoints.
- 3) La dependencia de deteccion YOLO es opcional en inferencia, pero necesaria para variantes guiadas por boxes.

V. PROTOCOLO EXPERIMENTAL

A. Estrategia de Evaluacion Local

La evaluacion local se ejecuta en inferencia con split por `bundle_asset_id` controlado por `infer.val_ratio`. Segun entrypoint, se reportan Hit@K/Recall@K para `infer.eval_ks`:

- 1) `src.infer` (pipeline principal).
- 2) `src.new_infer` (v8).
- 3) `src.infer_phase1`.
- 4) `src.infer_top1`.

B. Artefactos de Salida

Artefactos frecuentes del flujo train/infer:

- `metrics.jsonl`, `best.pt`, `epoch_X.pt` en entrenamiento.
- `test_submission.csv` y `val_metrics.json` para `src.infer`.
- `test_submission_v8.csv`, `val_metrics_v8.json` para `src.new_infer`.
- `test_submission_phase1.csv`, `val_metrics_phase1.json` para `src.infer_phase1`.
- `test_submission_top1.csv`, `val_metrics_top1.json` para `src.infer_top1`.

C. Reportes Analiticos Observados en el Workspace

`outputs/ts_date_alignment_report.csv` (1,876 bundles):

- 6,493 productos enlazados con timestamp disponible.
- 12.26% de productos comparten fecha exacta bundle-producto.
- 54.57% comparten mes.
- 81.01% comparten trimestre.

`outputs/category_section_summary.json`:

- 88 categorias totales en joins train.
- Para categorias con al menos 30 muestras: 9 son single-section y 10 casi single-section (umbral de pureza 0.95).

VI. REPRODUCIBILIDAD

A. Entorno

Dependencias base en `requirements.txt`: `pandas`, `numpy`, `pillow`, `tqdm`, `requests`, `hydra-core`, `torch`, `torchvision`.

Dependencias adicionales usadas por pipelines avanzados:

- `open_clip_torch` (entrenamiento/inferencia OpenCLIP).
- `ultralyticsplus` (deteccion YOLO clothing).
- `faiss` (opcional, aceleracion de busqueda en `src.new_infer.py`).

B. Determinismo

El codigo fija semillas de `random`, `numpy` y `torch`. `offline_augment.py` usa semillas estables por hash de `asset_id`+indice, permitiendo regenerar exactamente las mismas vistas sinteticas.

C. Configuracion Hydra

Se centraliza en:

- `config/config.yaml`: hiperparametros de train/infer.
- `config/files/file.yaml`: rutas de datos/imagenes/cache.

Los runs Hydra crean directorios de salida trazables por fecha/hora para comparar experimentos.

D. Precondicion de Entrenamiento

El entrypoint `src.train.py` apunta por defecto a `data/product_dataset_with_gender.csv`; por tanto, para reproducir train completo es necesario ejecutar antes `python src/utils/add_gender.py`.

E. Contrato de Entrega

El CSV final usa columnas:

- `bundle_asset_id`
- `product_asset_id`

La evaluacion considera el orden de ranking y hasta 15 predicciones por bundle.

VII. LIMITACIONES Y RIESGOS

- 1) **Supervision visual limitada por producto:** muchos IDs tienen una sola vista, lo que dificulta robustez ante pose, oclusion e iluminacion.
- 2) **Val por defecto en train entrypoint:** `src/train.py` construye train/val desde el mismo CSV de relaciones; para seleccion estricta de modelo conviene separar manifiestos de validacion.
- 3) **Dependencia de modelos externos:** OpenCLIP y YOLO requieren pesos externos y pueden introducir variabilidad de entorno.
- 4) **Ausencia de suite de tests automatizados:** el repositorio no incluye tests unitarios/integracion activos para proteger regresiones.
- 5) **Calibracion de submission:** no todas las variantes fuerzan exactamente 15 filas por bundle, por lo que la eleccion del script afecta formato y cobertura de ranking.

Lineas de mejora recomendadas: mejor separacion train/val, suite de tests, reranking aprendido adicional y consolidacion de dependencias opcionales en instalacion reproducible.

VIII. CONCLUSIONES

El repositorio Hack2026 dispone de un stack completo para retrieval bundle→product: curacion de datos, aumento offline, entrenamiento OpenCLIP, multiples estrategias de inferencia y generacion de entregas en formato de competicion.

La implementacion actual prioriza trazabilidad operativa (Hydra, checkpoints, metricas y reportes CSV/JSON) y permite iterar rapidamente sobre variantes de ranking y post-procesado sin rehacer el pipeline de base.

Este documento queda como referencia tecnica del estado vigente del sistema y como base para trabajo futuro en

validacion rigurosa, calibracion del ranking y optimizacion de latencia en produccion.

REFERENCES

APPENDIX

APENDICE: COMANDOS REPRODUCIBLES

A. Instalacion

```
pip install -r requirements.txt
pip install open_clip_torch ultralyticsplus
# opcional: pip install faiss-cpu
```

B. Descarga de Imagenes

```
bash download.sh
```

C. Preprocesamiento

```
python preprocess_data.py \
  --skip_download \
  --out_dir data \
  --val_ratio 0.1 \
  --seed 42
```

D. Generacion de Metadata de Genero

```
python src/utils/add_gender.py
```

E. Entrenamiento (Hydra)

```
python -m src.train
```

F. Inferencia Principal

```
python -m src.infer infer.checkpoint_path=<ruta_a_best.pt>
```

G. Inferencia v8

```
python -m src.new_infer infer.checkpoint_path=<ruta_a_best.pt>
```

H. Inferencia Phase 1

```
python -m src.infer_phase1 infer.checkpoint_path=<ruta_a_best.pt>
```

I. Inferencia Top-1

```
python -m src.infer_top1 infer.checkpoint_path=<ruta_a_best.pt>
```

J. Script Legacy OpenCLIP

```
python -m src.infer_openclip \
  --checkpoint outputs/retrieval_openclip/best.pt \
  --submission-out outputs/retrieval_openclip/submission.csv
```

K. Reporte de Timestamps

```
python src/utils/check_link_timestamps.py \
  --out-csv outputs/ts_date_alignment_report.csv \
  --timezone UTC
```